Introduction to GPU programming: Architectures, memory and management.

GPU programming in computational engineering

Escuela de Doctorado 2025/26
University of Zaragoza

Sergio Martínez-Aranda sermar@unizar.es
Mario Morales-Hernández mmorales@unizar.es
Fluid Dynamics Tecnologies - I3A, University of Zaragoza

Universidad Zaragoza 1542
Escuela de Ingeniería y Arquitectura Universidad Zaragoza
Instituto Universitario de Investigación en Ingeniería de Aragón Universidad Zaragoza

Module content

1. Introduction to GPU architecture
2. Basic concepts for CUDA programming
3. First steps in CUDA programming

GPUs and parallel computing

- In the **1990s**, companies like NVIDIA and AMD pioneered the development of dedicated graphics cards. **Accelerate rendering for computer graphics** for video games and visual simulations.
- In the **2000s**, **programmable shaders** allowed GPUs to execute custom code for rendering effects.
- In **2006**, NVIDIA launched **CUDA (Compute Unified Device Architecture)**, a programing platform that enabled developers to use GPUs for **general-purpose computing (GPGPU)**. CUDA unlocked the massive parallel processing power of GPUs, making them ideal for tasks like scientific simulations, data analysis, and machine learning.
- In the **2010s**, GPU-accelerated computational models appeared for solving geophysical, hydraulic and atmospheric flows in the Earth surface. Computing simultaneously in multiple GPU devices (**High-Performance-Computing or HPC**) allows to solve very-large-scale problems with affordable simulation times.
- **Nowadays**, GPUs are the **cutting-edge technology for scientist computing**, AI research, cryptocurrency mining, data centers, autonomous vehicles, and even quantum computing simulations. **Cloud computing** has further enabled GPU access to researchers and developers without owning physical hardware.

(NVIDIA GeForce 256, 1999)

(NVIDIA GeForce RTX 5090, 2025)

GPU architecture vs CPU Architecture

CPU (Central Processing Unit)

- Designed for general-purpose computing and **sequential task execution**.
- Focuses on **low-latency processing** and optimizing for single-threaded performance.
- Features **fewer, more powerful cores** with complex control logic and large caches to handle diverse tasks efficiently.

(AMD Ryzen Threadripper PRO 7995WX, 2023)

GPU (Graphics Processing Unit)

- Specialized for **parallel processing** and high-throughput computations.
- Optimized for handling **thousands of lightweight threads simultaneously**.
- Comprises **thousands of smaller, simpler cores** (CUDA cores in NVIDIA GPUs or Stream Processors in AMD GPUs) designed for specific tasks like rendering graphics or performing matrix operations.

(NVIDIA GeForce RTX 5090, 2025)

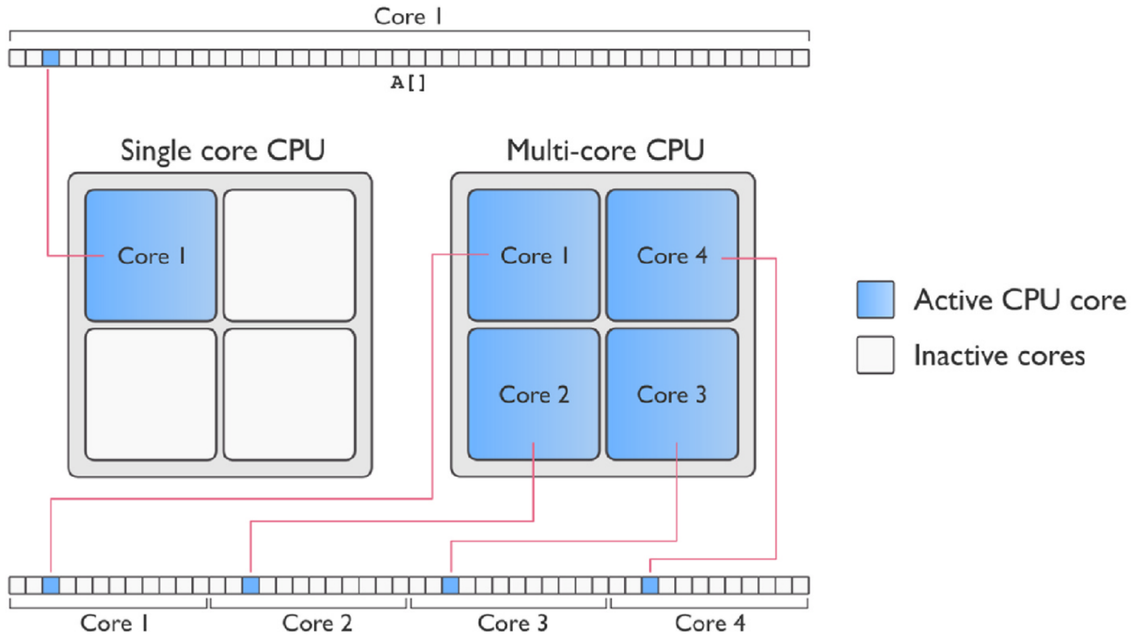
CPU Architecture

Single-thread execution

- Single Instruction, Single Data (SISD): Each task waits for the completion of the previous task, i.e. sequential processing.

Simultaneous Multi-Threading (SMT)

- Physical core has a full set of execution units and caches
- Physical cores are divided into two logical cores for the operating system, which shares the core resources.
- Single Instruction, Multiple Data (SIMD): Tasks are divided and executed simultaneously across multiple threads in a multi-core CPU node with shared memory.



(Reproduced from Fernández-Pato, 2025)

CPU parallelism

Single-thread execution

- Each task waits for the completion of the previous task, i.e. sequential processing.

Thread-based parallelism (OpenMP)

- Tasks are divided and executed simultaneously across multiple threads in a multi-core CPU node.
- Shared memory, i.e. all threads can access the same memory.
- Easy to implement with compiler directives, such as `#pragma omp`.

Process-based parallelism (Message Passing Interface)

- The problem can be divided into smaller sub-problems and they are solved simultaneously in independent CPU nodes.
- Distributed memory, i.e. each sub-problem accesses its own memory.
- Scalable performance in clusters but it requires explicit message passing between sub-problems.

GPU architecture

Thread

- The thread is the smallest unit of execution.
- Thousands of threads can run simultaneously on the GPU kernels.

Block

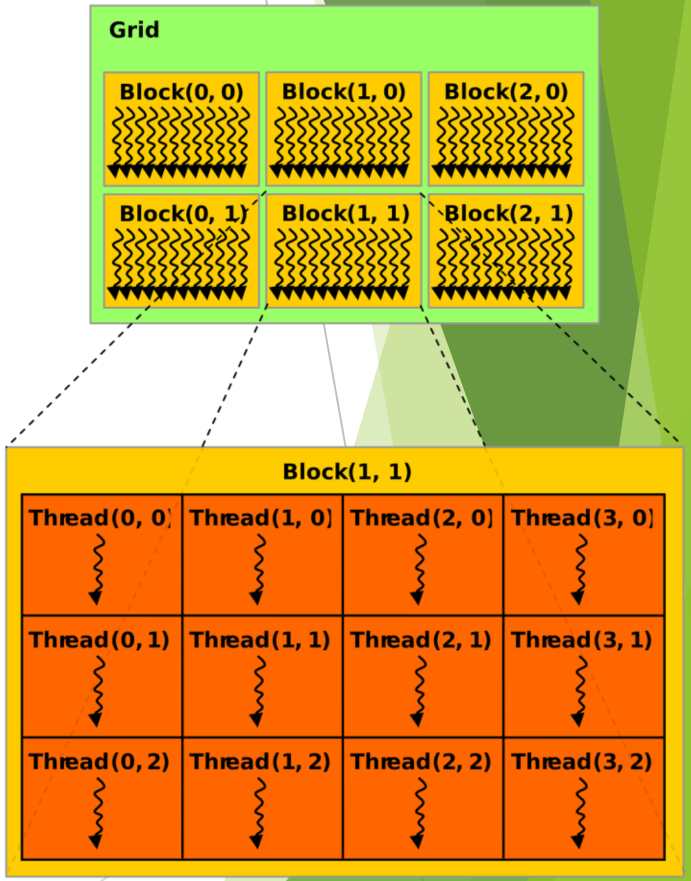
- Threads are grouped into blocks.
- Collections of threads that can synchronize and communicate using shared memory.

Grid

- The problem is divided into a grid, i.e. collection of blocks that execute the same kernel function.
- Threads in a grid can access to the same (global) memory.

Specialized Units

- Dedicated units for specific tasks, such as RT cores for ray tracing or Tensor cores for AI acceleration.



(Reproduced from https://maori.geek.nz, 2021)

GPU parallelism

SIMT (Single Instruction, Multiple Threads):

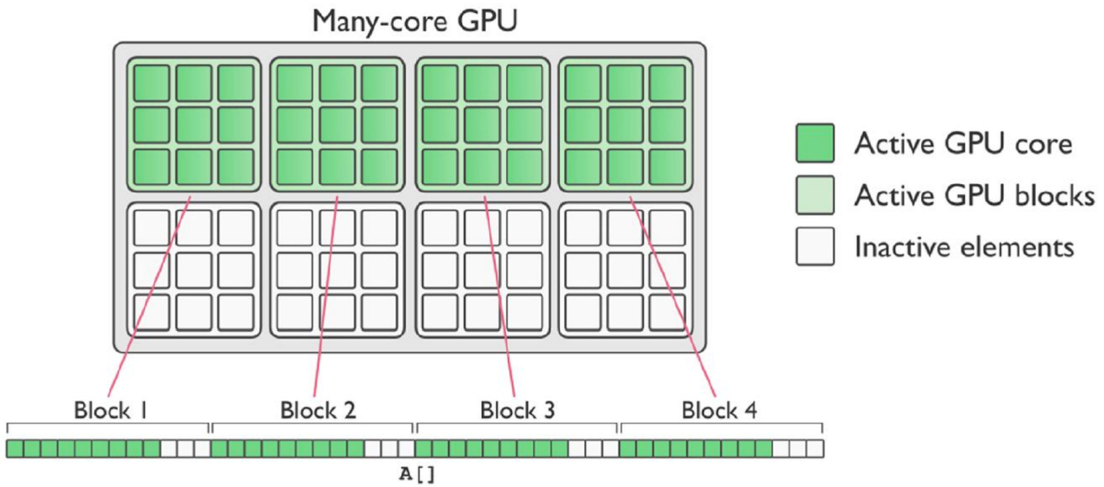
- Multiple threads execute the same instruction on different data points simultaneously.
- Blocks are scheduled in Streaming Multiprocessors (SMs).
- Threads run in warps, i.e. a group of 32 threads (NVIDIA GPUs) that execute together lockstep.

Thread divergence:

- Threads can diverge, i.e. follow different execution paths with different execution times.
- This reduces efficiency.

Memory coalescing:

- Threads access to consecutive memory locations.



(Reproduced from Fernández-Pato, 2025)

GPU development platforms

There are several GPU development platforms available, each with its own strengths and target use cases.

- **CUDA:** The most widely used platform, especially in for scientific computing and AI.
- **OpenCL:** Popular for cross-platform and heterogeneous computing.
- **HIP:** Gaining traction for AMD GPUs and CUDA compatibility.
- **SYCL:** Emerging as a modern C++ alternative for heterogeneous computing.

CUDA platform

https://developer.nvidia.com/

- Developer: NVIDIA
- Platform Support: Primarily NVIDIA GPUs
- Ease of Use: More user-friendly, extensive documentation, and mature ecosystem
- Performance: Optimized for NVIDIA hardware, often faster on NVIDIA GPUs
- API and Ecosystem: Rich ecosystem with libraries (cuBLAS, cuDNN, etc.) and tools (Nsight)
- Portability: Limited to NVIDIA GPUs
- Adoption: Widely used in scientific computing, AI and deep learning.
- Programming Model: SIMT (Single Instruction, Multiple Threads)
- Community Support: Large community, strong industry support

Module content

1. Introduction to GPU architecture
2. Basic concepts for CUDA programming
3. First steps in CUDA programming

CUDA/C++ platforms

The NVIDIA® CUDA® Toolkit provides a comprehensive development environment for C/C++ developers building GPU-accelerated applications.

NVIDIA driver: Enable the operating system and applications to communicate effectively with NVIDIA GPUs.

Compiler: The NVIDIA CUDA Compiler (nvcc) which compiles CUDA/C++ code into applications executable on GPUs.

Runtime: The CUDA API provides functions to manage devices, memory, and execute kernels.

Developing tools: Tools for debugging, profiling, and optimizing CUDA applications.

- nvidia-smi
- compute-sanitizer

Libraries: A set of highly optimized libraries for various domains.

- cuBLAS library is an implementation of Basic Linear Algebra Subprograms (BLAS) on the NVIDIA CUDA runtime.
- cuSPARSE library contains subroutines for handling sparse matrices implemented on the NVIDIA CUDA runtime.

Additional tools: nvtop, VScode, etc.

CUDA/C++ code workflow

GPU-accelerated apps include **parallelized CUDA kernels** running into a **sequential workflow** running in the CPU core.

```

// Driven fucntion exected in CPU
int main()
{
    ...
    RAM memory allocation and initialization
    ...
    GPU selection and setup
    ...
    CUDA global memory allocation
    ...
    RAM --> CUDA data transfer
    ...
    // Kernel invocation with multi-threads
    say_hi <<<numBlocks, threadsPerBlock>>> ();
    ...
    CUDA --> RAM memory transfer
    ...
    Write output files
    ...
}

```

Sequential workflow

CUDA kernel parallelized in the GPU device.

Loop execution

```
A=A[<1>,...,<1024>]
for i=1,...,1024
    A[i]=...
end for
```

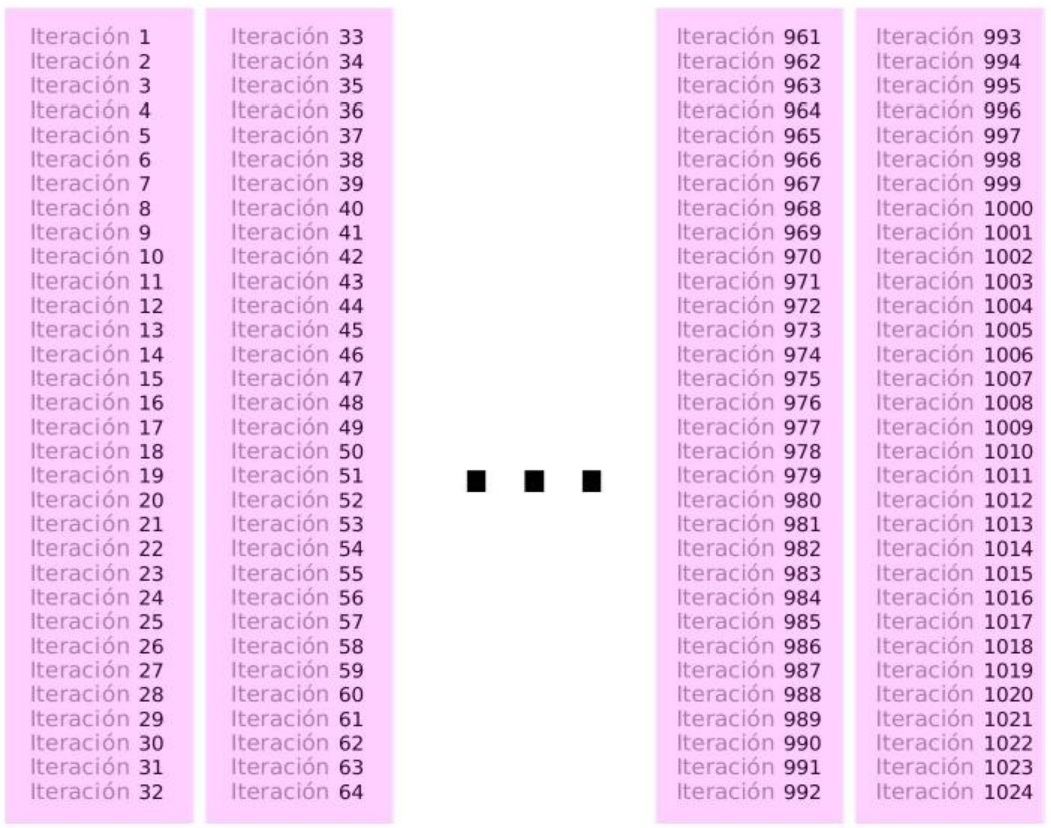
- The whole 1024 **operations** must be executed in parallel.
- Each operation is executed by a **thread**.

Iteración 1	Iteración 33		Iteración 961	Iteración 993
Iteración 2	Iteración 34		Iteración 962	Iteración 994
Iteración 3	Iteración 35		Iteración 963	Iteración 995
Iteración 4	Iteración 36		Iteración 964	Iteración 996
Iteración 5	Iteración 37		Iteración 965	Iteración 997
Iteración 6	Iteración 38		Iteración 966	Iteración 998
Iteración 7	Iteración 39		Iteración 967	Iteración 999
Iteración 8	Iteración 40		Iteración 968	Iteración 1000
Iteración 9	Iteración 41		Iteración 969	Iteración 1001
Iteración 10	Iteración 42		Iteración 970	Iteración 1002
Iteración 11	Iteración 43		Iteración 971	Iteración 1003
Iteración 12	Iteración 44		Iteración 972	Iteración 1004
Iteración 13	Iteración 45		Iteración 973	Iteración 1005
Iteración 14	Iteración 46		Iteración 974	Iteración 1006
Iteración 15	Iteración 47		Iteración 975	Iteración 1007
Iteración 16	Iteración 48		Iteración 976	Iteración 1008
Iteración 17	Iteración 49		Iteración 977	Iteración 1009
Iteración 18	Iteración 50		Iteración 978	Iteración 1010
Iteración 19	Iteración 51	...	Iteración 979	Iteración 1011
Iteración 20	Iteración 52		Iteración 980	Iteración 1012
Iteración 21	Iteración 53		Iteración 981	Iteración 1013
Iteración 22	Iteración 54		Iteración 982	Iteración 1014
Iteración 23	Iteración 55		Iteración 983	Iteración 1015
Iteración 24	Iteración 56		Iteración 984	Iteración 1016
Iteración 25	Iteración 57		Iteración 985	Iteración 1017
Iteración 26	Iteración 58		Iteración 986	Iteración 1018
Iteración 27	Iteración 59		Iteración 987	Iteración 1019
Iteración 28	Iteración 60		Iteración 988	Iteración 1020
Iteración 29	Iteración 61		Iteración 989	Iteración 1021
Iteración 30	Iteración 62		Iteración 990	Iteración 1022
Iteración 31	Iteración 63		Iteración 991	Iteración 1023
Iteración 32	Iteración 64		Iteración 992	Iteración 1024

Loop execution

```
A=A[<1>,...,<1024>]
for i=1,...,1024
    A[i]=...
end for
```

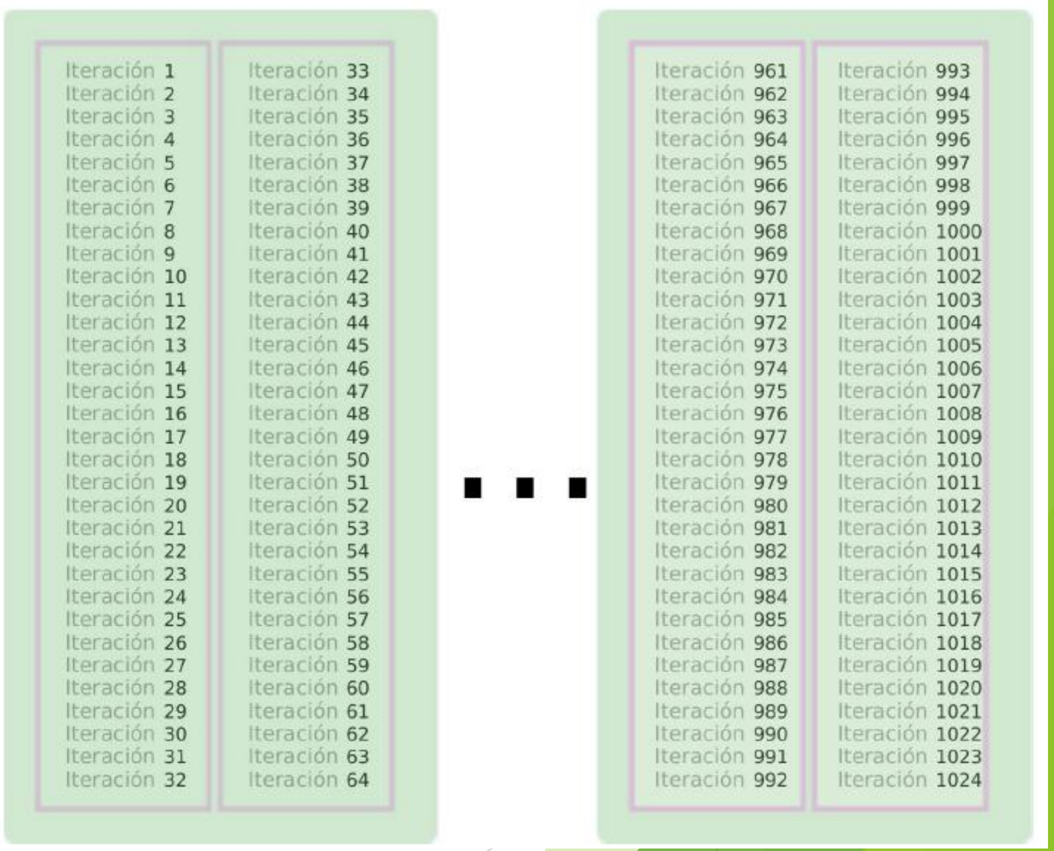
- The whole 1024 **operations** must be executed in parallel.
- Each operation is executed by a **thread**.
- Sets of 32 threads are established → **warp**



Loop execution

```
A=A[<1>,...,<1024>]
for i=1,...,1024
    A[i]=...
end for
```

- The whole 1024 **operations** must be executed in parallel.
- Each operation is executed by a **thread**.
- Sets of 32 threads are established → **warp**
- Warps are grouped in sets → **blocks**



Loop execution

```
A=A[<1>,...,<1024>]
for i=1,...,1024
    A[i]=...
end for
```

- The whole 1024 **operations** must be executed in parallel.
- Each operation is executed by a **thread**.
- Sets of 32 threads are established called **warp**
- Warps are grouped in sets, known as **blocks**
- All the blocks are included into the **grid**

Loop execution

```
A=A[<1>,...,<1024>]
for i=1,...,1024
    A[i]=...
end for
```

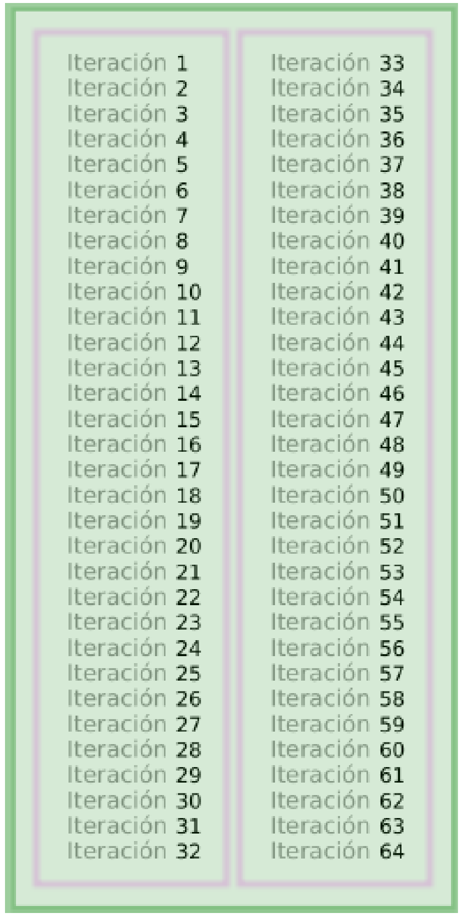
- The **grid** contains all the operations to be processed in the GPU.

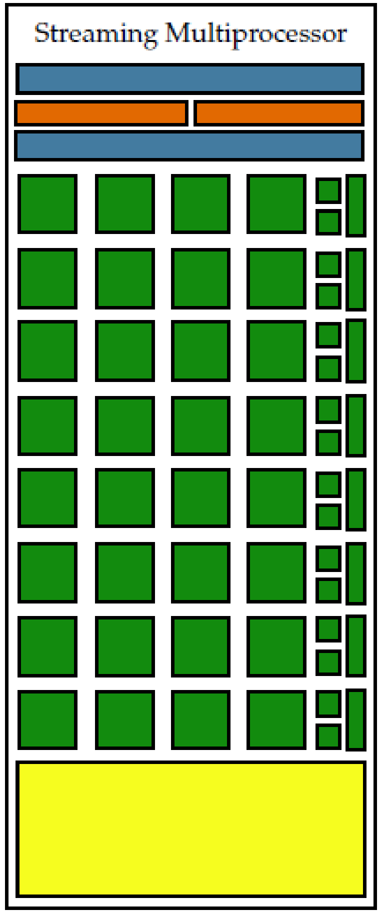
- The loop is executed by a **kernel** function.

Loop execution

```
A=A[<1>,...,<1024>]
for i=1,...,1024
    A[i]=...
end for
```

- The **grid** contains all the operations to be processed in the GPU.
- The loop is executed by a **kernel** function.
- **Blocks** are addressed by the **Streaming Multiprocessors** (SM/SMx).

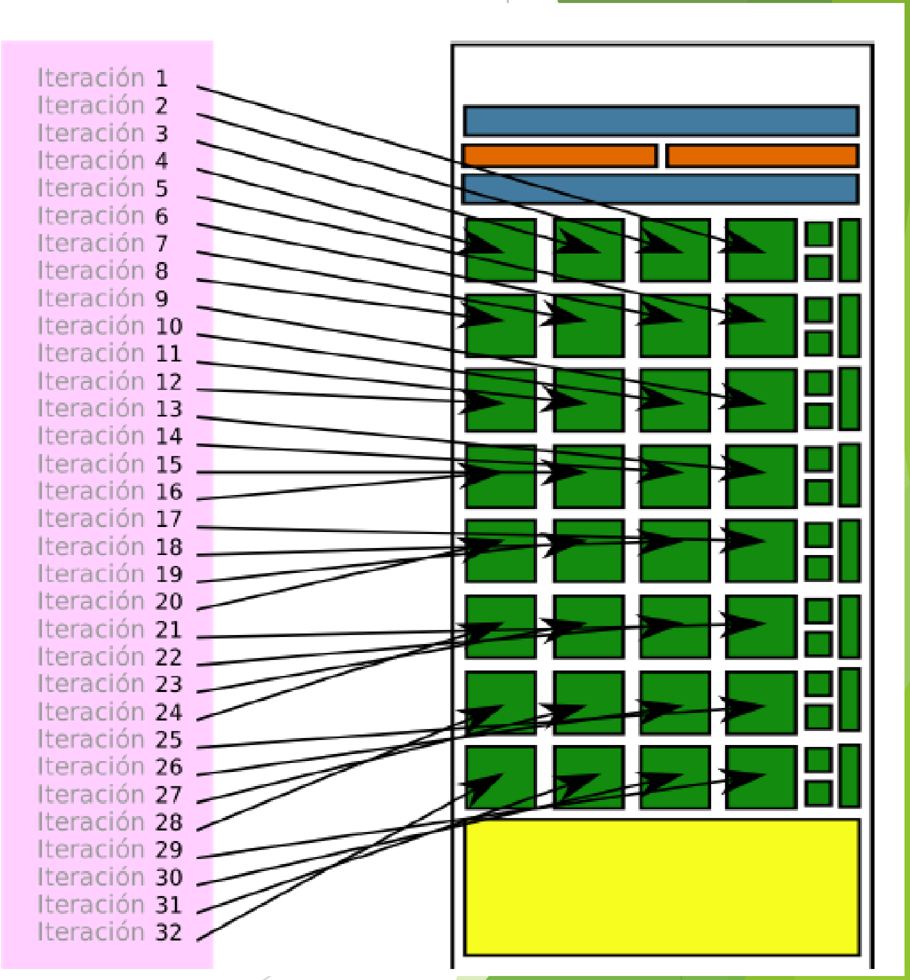




Loop execution

```
A=A[<1>,...,<1024>]
for i=1,...,1024
    A[i]=...
end for
```

- The **grid** contains all the operations to be processed in the GPU.
- The loop is executed by a **kernel** function.
- **Blocks** are addressed by the **Streaming Multiprocessors** (SM/SMx).
- **Warps** (32 threads) in each block are executed sequentially in the SM.



CUDA/C++ parallel kernel

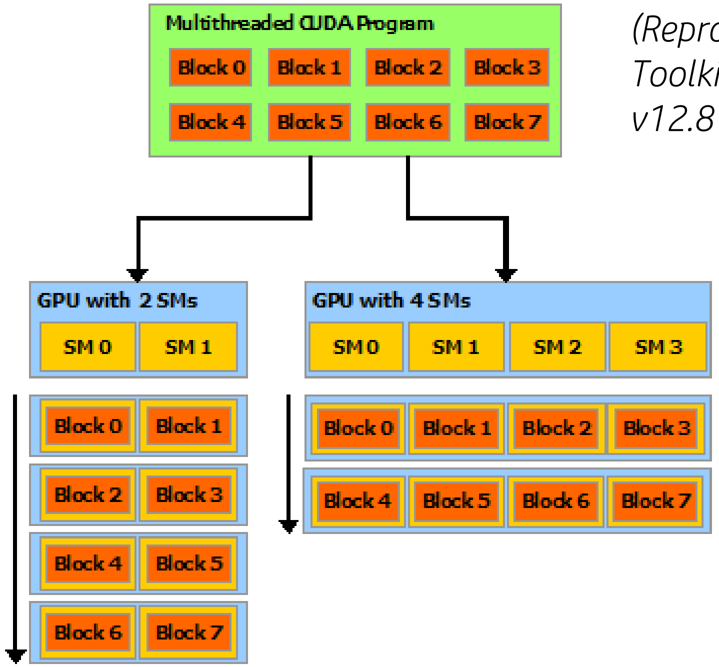
- The multi-threaded kernel is partitioned into **thread blocks**, that execute independently from each other.
- Thread blocks are scheduled in an array of **Streaming Multiprocessors** (SMs).
- GPUs with more multiprocessors execute the program in less time than a GPU with fewer multiprocessors.

```

// Kernel definition
__global__ void say_hi()
{
    int idx = (blockIdx.x * blockDim.x) + threadIdx.x;
    printf("Hi from Thread %d\n", idx);
}

// Driven fucntion exected in CPU
int main()
{
    ...
    // Kernel invocation with multi-threads
    say_hi <<<numBlocks, threadsPerBlock>>> ();
    ...
}

```



(Reproduced from CUDA Toolkit Documentation v12.8 Update 1, 2025)

CUDA/C++ code workflow

GPU-accelerated apps include **parallelized CUDA kernels** running into a **main driven function** running in the CPU core.

```

// Driven fucntion exected in CPU
int main()
{
    ...
    RAM memory allocation and initialization
    ...
    GPU selection and setup
    ...
    CUDA global memory allocation
    ...
    RAM --> CUDA data transfer
    ...
    // Kernel invocation with multi-threads
    say_hi <<<numBlocks, threadsPerBlock>>> ();
    ...
    CUDA --> RAM memory transfer
    ...
    Write output files
    ...
}

```

Sequential workflow

RAM memory allocation is required.

CUDA memory structure is the key factor to achieve high performance in the CUDA kernel execution.

Barriers for thread synchronization and memory transfers RAM-GPU represent critical points for the code performance.

GPU memory hierarchy

RAM memory (CPU)

- Large memory shared across all the CPU cores.
- Optimized for random access patterns and complex control flows.
- Primary working memory for I/O communications.

Global Memory

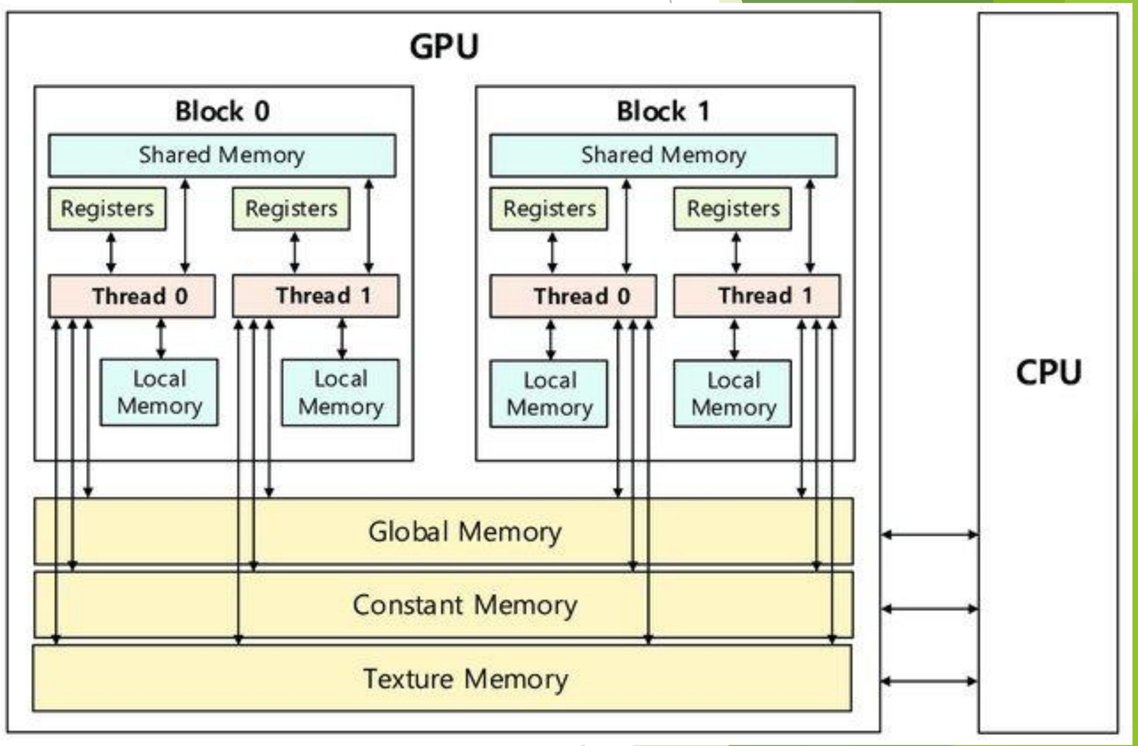
- High-capacity memory but with higher latency.
- Accessible by all the threads in the grid.

Shared Memory

- Faster, on-chip memory shared among the treads of each blocks.
- Limited to 48KB per block in most of GPUs.

Registers

- Ultra-fast memory dedicated to individual threads.
- Limited to 255 registers per thread.

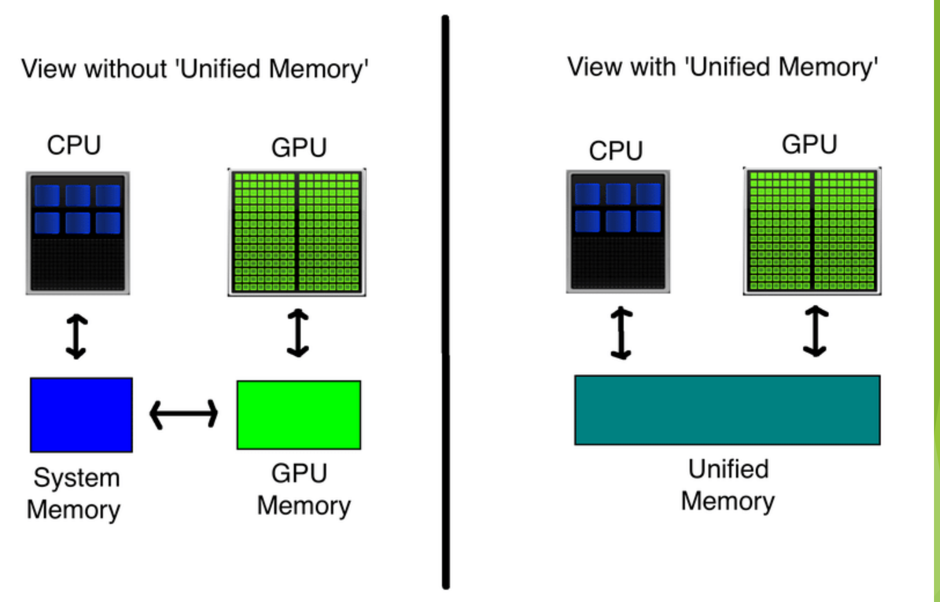


(Reproduced from An and Seo, 2020)

GPU memory hierarchy

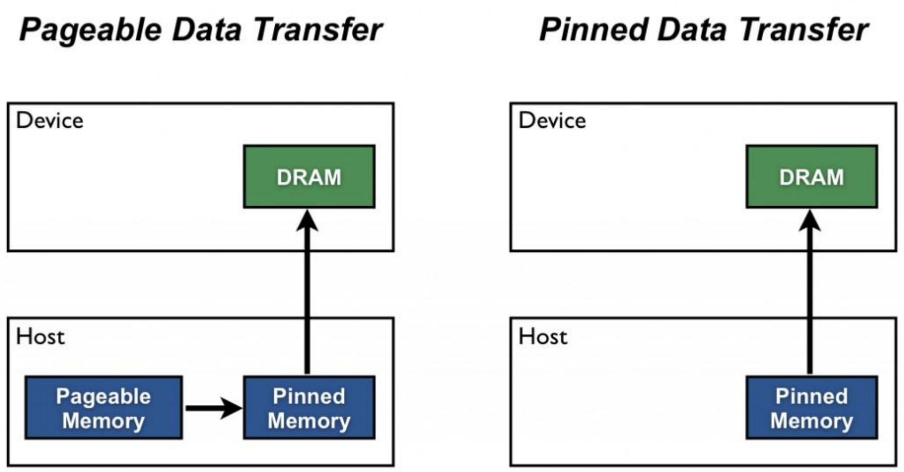
Unified Memory

- Shared memory space between CPU and GPU.
- Simplifies programming by removing explicit memory copies.
- Useful for irregular memory access patterns.
- May introduce overhead due to automatic data migration.



Pinned Memory

- Host memory locked in physical memory.
- Enables faster data transfers between CPU and GPU.
- Supports asynchronous data transfers.
- Ideal for frequent and large data transfers.



GPU memory hierarchy

Memory Type	Access Scope	Latency	Bandwidth	Use Case
Registers	Private to each thread	Low	High	Temporary variables for individual threads
Shared Memory	Within a thread block	Low	High	Data shared among threads in the same block
Global Memory	Accessible by all threads	High	Moderate	Large datasets shared across threads
Texture/Constant	Read-only by all threads	Moderate	High	Frequently accessed data with spatial locality
Unified Memory	Shared by CPU and GPU	Moderate	Moderate	Simplifies programming by removing explicit memory copies
Pinned Memory	Host memory (CPU)	Low (transfer)	High (transfer)	Faster data transfers between CPU and GPU

CPU-GPU memory transfer

Transfer Methods

- Explicit copy (cudaMemcpy, hipMemcpy, etc) Global memory / RAM memory
- Pinned Memory for faster transfers
- Asynchronous transfers with streams to overlap computation
- Unified Memory for simplified automatic management

Best Practices

- Memory transfers are often the bottleneck in GPU applications
- Minimize CPU-GPU transfers
- Batch small transfers together
- Use registers/shared memory for frequently accessed data
- Coalesce global memory accesses when possible
- Balance register usage vs occupancy
- Profile memory access patterns to identify bottlenecks

CUDA example

Standard C Code

```
void saxpy(int n, float a,
           float *x, float *y)
{
  for (int i = 0; i < n; ++i)
    y[i] = a*x[i] + y[i];
}

int N = 1<<20;



// Perform SAXPY on 1M elements
saxpy(N, 2.0, x, y);
```

C with CUDA extensions

```
__global__
void saxpy(int n, float a,
           float *x, float *y)
{
  int i = blockIdx.x*blockDim.x + threadIdx.x;
  if (i < n) y[i] = a*x[i] + y[i];
}

int N = 1<<20;
cudaMemcpy(x, d_x, N, cudaMemcpyHostToDevice);
cudaMemcpy(y, d_y, N, cudaMemcpyHostToDevice);

// Perform SAXPY on 1M elements
saxpy<<<4096,256>>>(N, 2.0, x, y);

cudaMemcpy(d_y, y, N, cudaMemcpyDeviceToHost);
```

CUDA example

Standard C Code

```c
void saxpy(int n, float a,
           float *x, float *y)
{
  for (int i = 0; i < n; ++i)
    y[i] = a*x[i] + y[i];
}

int N = 1<<20;

// Perform SAXPY on 1M elements
saxpy(N, 2.0, x, y);
```

C with CUDA extensions

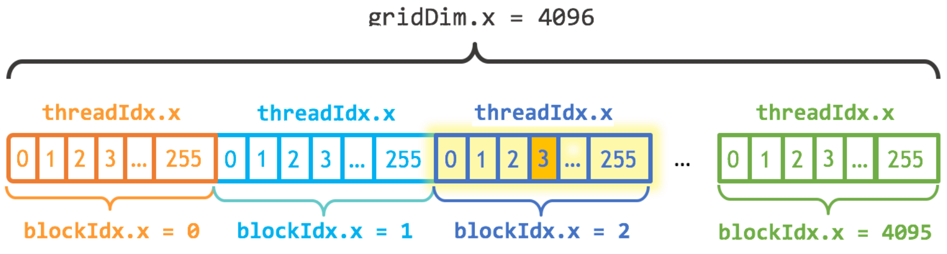
```c
__global__
void saxpy(int n, float a,
           float *x, float *y)
{
  int i = blockIdx.x*blockDim.x + threadIdx.x;
  if (i < n) y[i] = a*x[i] + y[i];
}

int N = 1<<20;
cudaMemcpy(x, d_x, N, cudaMemcpyHostToDevice);
cudaMemcpy(y, d_y, N, cudaMemcpyHostToDevice);

// Perform SAXPY on 1M elements
saxpy<<<4096,256>>>(N, 2.0, x, y);

cudaMemcpy(d_y, y, N, cudaMemcpyDeviceToHost);
```

- Each thread can establish its unique identifier within the block.
- Each block can establish its identifier within the grid.




```
index = blockIdx.x * blockDim.x + threadIdx.x

index = (2) * (256) + (3) = 515
```

CUDA example

Standard C Code

```
void saxpy(int n, float a,
           float *x, float *y)
{
  for (int i = 0; i < n; ++i)
    y[i] = a*x[i] + y[i];
}

int N = 1<<20;


// Perform SAXPY on 1M elements
saxpy(N, 2.0, x, y);
```

C with CUDA extensions

```
__global__
void saxpy(int n, float a,
           float *x, float *y)
{
  int i = blockIdx.x*blockDim.x + threadIdx.x;
  if (i < n) y[i] = a*x[i] + y[i];
}

int N = 1<<20;
cudaMemcpy(x, d_x, N, cudaMemcpyHostToDevice);
cudaMemcpy(y, d_y, N, cudaMemcpyHostToDevice);

// Perform SAXPY on 1M elements
saxpy<<<4096,256>>>(N, 2.0, x, y);

cudaMemcpy(d_y, y, N, cudaMemcpyDeviceToHost);
```

- The kernels are executed N times in parallel by N different CUDA threads
- A kernel is defined using the __global__ declaration
- The number of CUDA threads that execute that kernel for a given kernel call is specified using a <<< … >>> execution configuration syntax.

CUDA example

Standard C Code

```
void saxpy(int n, float a,
           float *x, float *y)
{
  for (int i = 0; i < n; ++i)
    y[i] = a*x[i] + y[i];
}

int N = 1<<20;



// Perform SAXPY on 1M elements
saxpy(N, 2.0, x, y);
```

C with CUDA extensions

```
__global__
void saxpy(int n, float a,
           float *x, float *y)
{
  int i = blockIdx.x*blockDim.x + threadIdx.x;
  if (i < n) y[i] = a*x[i] + y[i];
}

int N = 1<<20;
cudaMemcpy(x, d_x, N, cudaMemcpyHostToDevice);
cudaMemcpy(y, d_y, N, cudaMemcpyHostToDevice);

// Perform SAXPY on 1M elements
saxpy<<<4096,256>>>(N, 2.0, x, y);

cudaMemcpy(d_y, y, N, cudaMemcpyDeviceToHost);
```

- Explicit CPU-GPU memory transfer before and after the CUDA kernel

CUDA/C++ compilation and execution

- CUDA/C++ code is written in `app_code.cu` files, including the header:

```
#include <cuda_runtime.h>
```

- NVCC compiler is used to create executable files:

```
nvcc -arch=sm_XX -O3 -G -o exe_app app_code.cu -lXXXXX
```

- Additional flags for `nvcc` compiler:

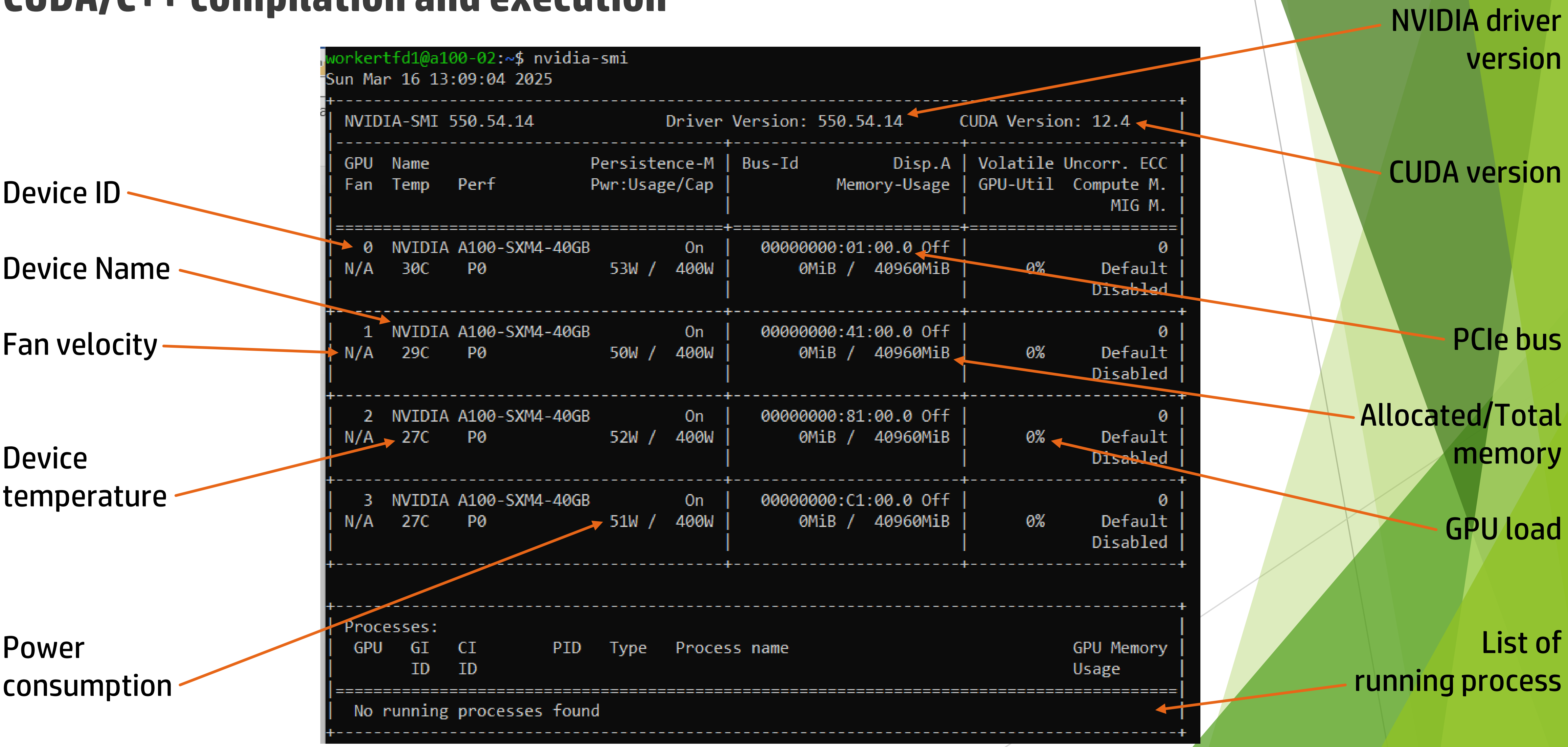
`-arch=sm_XX` to specify the compute capability of the GPU device
`-G` to include memory check information for debugging tools.
`-O3` to enable high-level optimizations
`-lXXXXX` to include external libraries, such as `-lcudart` or `-lcublas`.

- The compiled application runs with the command:

```
./exe_app
```

- `nvidia-smi` command provides performance data of the GPU during the execution of the application.

CUDA/C++ compilation and execution



Module content

1. Introduction to GPU architecture
2. Basic concepts for CUDA programming
3. First steps in CUDA programming

Working in the HERMES supercomputing cluster

HERMES supercomputing cluster is the scientific infrastructure dedicated for researchers at the Institute of Engineering Research (I3A), University of Zaragoza. HERMES has over 3,260 CPU cores, with 5,920 parallel processing threads and 26 TB of RAM. HERMES features 251,392 CUDA cores (including NVIDIA A100 GPUs) and 720 GB of dedicated memory.

Instituto Universitario de Investigación en Ingeniería de Aragón
Universidad Zaragoza

CUDA hands-on session during W2

Dedicated computation node with 4 x NVIDIA A100 devices.
20 working user profiles: 4 groups of 5 users each.

- Remote access by Ubuntu terminal:
```
ssh workertfd<X>@155.210.134.18 -p 4001
```

- User: workertfd<X> with X=2,...,20
  Note: workertfd1 is reserved for professors.

- Password: Workertfd2026

Main step in CUDA/C++ code workflow

Sequential workflow

GPU selection and setup

CUDA memory allocation

CPU-GPU data transfer

CUDA kernel execution

GPU selection and setup

```
int numGPUs = 0;
cudaGetDeviceCount(&numGPUs);
```

Get the number of available GPUs with computing capabilities.

```
int gpu_id = 0;
cudaDeviceProp props;
cudaGetDeviceProperties(&props, gpu_id);
```

Get the specific properties of the selected GPU device. The input gpu_id is type integer. The output is class-type cudaDeviceProp.

GPU selection and setup

```
int numGPUs = 0;
cudaGetDeviceCount(&numGPUs);
```

Get the number of available GPUs with computing capabilities.

```
int gpu_id = 0;
cudaDeviceProp props;
cudaGetDeviceProperties(&props, gpu_id);
```

Get the specific properties of the selected GPU device. The input gpu_id is type integer. The output is class-type cudaDeviceProp.

```
int selectedGPU = 0;
cudaSetDevice(selectedGPU);
```

Select a specific GPU device to run the parallel CUDA kernels.

```
size_t freeMemory, totalMemory;
cudaMemGetInfo(&freeMemory, &totalMemory);
```

Get the total available memory and the free memory for a specific GPU device.

```
cudaDeviceSynchronize();
```

It ensures all the CUDA tasks on the GPU device have been completed before the host (CPU) continues execution.

Main step in CUDA/C++ code workflow

Sequential workflow

GPU selection and setup

CUDA memory allocation

CPU-GPU data transfer

CUDA kernel execution

CUDA memory allocation: Global memory

```

int main() {

    // Host memory
    size_t size = N*sizeof(double);
    double *h_A;
    h_A = (double*) malloc(size);

    ...
    double *d_A;
    cudaMalloc((void**) &d_A, size);


    int threadsPerBlock = 256;
    int blocksPerGrid = N/threadsPerBlock+1;

    memoryCheck
        <<<blocksPerGrid, threadsPerBlock>>>
        (N, size, d_A);


    cudaFree(d_A);
    ...
}

```

Dynamic allocation of computation arrays in the host memory

Dynamic allocation of computation arrays in the GPU global memory

Parallelized kernel executed in the CUDA grid

Free GPU global memory

CUDA memory allocation: Thread register memory

```
__global__ void registerMemoryAllocate(int N, size_t size, double *d_A) {

    double rA;

    int idx = ( blockIdx.x * blockDim.x ) + threadIdx.x;
    if (idx < N) {
        rA = (double)(idx);
        d_A[idx] = rA;
    }
}

int main() {
    ...
    int threadsPerBlock = 256;
    int blocksPerGrid = N/threadsPerBlock+1;

    registerMemoryAllocate
        <<<blocksPerGrid, threadsPerBlock>>>
        (N, size, d_A);
    ...
}
```

Static register memory allocation

Parallel computation of the register at each thread and output to the global memory array.

Parallelized kernel executed in the CUDA grid

CUDA memory allocation: Block shared memory

```
__global__ void sharedMemoryAllocate(int N, size_t size, double *d_A) {

    int ithread = threadIdx.x;
    int idx = ( blockIdx.x * blockDim.x ) + threadIdx.x;

    extern __shared__ double shared_A[];
    if (idx < N) {
        shared_A[ithread] = (double)(idx);
    }
    __syncthreads();

    if (idx < N) {
        d_A[idx] = shared_A[ithread];
    }
}

int main() {
    ...
    int threadsPerBlock = 256;
    int blocksPerGrid = N/threadsPerBlock+1;
    size_t requiredSharedMemory = threadsPerBlock*sizeof(double);

    sharedMemoryAllocate
        <<<blocksPerGrid, threadsPerBlock, requiredSharedMemory>>>
        (N, size, d_A);
    ...
}
```

Dynamical allocation of the shared memory. The idx thread computes to the ithread position in the shared array of the blockIdx.x block. __syncthreads() ensures that all threads have finished writing the shared array.

Parallel output to the global memory array.

Parallelized kernel executed in the CUDA grid. The size of the shared memory should be provided in the CUDA grid definition.

Main step in CUDA/C++ code workflow

Sequential workflow

GPU selection and setup

CUDA memory allocation

CPU-GPU data transfer

CUDA kernel execution

CPU-GPU data transfer

```
int main() {

    size_t size = N*sizeof(double); // arrays size

    double *h_A = (double*) malloc(size); // host array

    double *d_A;
    cudaMalloc((void**) &d_A, size); // device array

    ...
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);


    vectorAdd <<<blocksPerGrid, threadsPerBlock>>> (N, d_A, d_B);


    cudaMemcpy(h_B, d_B, size, cudaMemcpyDeviceToHost);
    ...
}
```

Copy array from host to global memory in the device

Copy array from device global memory back to host memory

CPU-GPU data transfer

```
int main() {

    size_t size = N*sizeof(double); // arrays size

    double *h_A = (double*) malloc(size); // host array

    double *d_A;
    cudaMalloc((void**) &d_A, size); // device array

    ...
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);


    vectorAdd <<<blocksPerGrid, threadsPerBlock>>> (N, d_A, d_B);


    cudaMemcpy(h_B, d_B, size, cudaMemcpyDeviceToHost);
    ...
}

```

Copy array from host to global memory in the device

Copy array from device global memory back to host memory

```

cudaMemset(d_B, 0, size);
cudaMemset(d_D, 0xFF, N*sizeof(int)); //integer array

```

Set global memory to specific values (byte level).

Main step in CUDA/C++ code workflow

Sequential workflow

GPU selection and setup

CUDA memory allocation

CPU-GPU data transfer

CUDA kernel execution

CUDA kernel execution: Grid setup

```
__global__ void arrayComputation(int N, double *A) {

    int idx = ( blockIdx.x * blockDim.x ) + threadIdx.x;
    if (idx < N) {
        A[idx] = ...
    }
}

int main() {

    int N = 1000;

    double *d_A;
    cudaMalloc((void**) &d_A, N*sizeof(double)); // device array

    ...
    arrayComputation <<<1, N>>> (N, d_A);



    int threadsPerBlock = 256;
    int blocksPerGrid = N/threadsPerBlock + 1;

    arrayComputation <<<blocksPerGrid, threadsPerBlock>>> (N, d_A);
    ...
}
```

__global__ function executed in the CUDA grid.

Number of tasks to execute in the parallel kernel, i.e. number of threads in the CUDA grid.

Single block execution.

CUDA kernel execution: Grid setup

```
__global__ void arrayComputation(int N, double *A) {

    int idx = ( blockIdx.x * blockDim.x ) + threadIdx.x;
    if (idx < N) {
        A[idx] = ...
    }
}

int main() {

    int N = 1000;

    double *d_A;
    cudaMalloc((void**) &d_A, N*sizeof(double)); // device array

    ...
    arrayComputation <<<1, N>>> (N, d_A);



    int threadsPerBlock = 256;
    int blocksPerGrid = N/threadsPerBlock + 1;

    arrayComputation <<<blocksPerGrid, threadsPerBlock>>> (N, d_A);
    ...
}
```

__global__ function executed in the CUDA grid.

Number of tasks to execute in the parallel kernel, i.e. number of threads in the CUDA grid.

Multiple block execution.

CUDA kernel execution: Device functions

```
__device__ void addDoubleData(double *A, double *B, double *C) {
    (*C) = (*A) + (*B);
}

__global__ void deviceVectorAdd(int N, double *A, double *B, double *C) {

    int idx = ( blockIdx.x * blockDim.x ) + threadIdx.x;
    if (idx < N) {
        addDoubleData( &(A[idx]), &(B[idx]), &(C[idx]) );
    }
}

int main() {

    int N = 1000;

    double *d_A;
    cudaMalloc((void**) &d_A, N*sizeof(double)); // device array

    ...
    int threadsPerBlock = 256;
    int blocksPerGrid = N/threadsPerBlock + 1;

    deviceVectorAdd <<<blocksPerGrid, threadsPerBlock>>> (N, d_A, d_B);
    ...
}
```

__device__ function for algebraic in-thread operations.

addDoubleData is executed in each thread.

Multiple blocks parallel kernel

GPU kernel execution: Coalescent memory access

```

__global__ void nonCoalescentVectorAdd(int N, double *A, double *B) {

    int idx = ( blockIdx.x * blockDim.x ) + threadIdx.x;
    if (idx < N) {
        A[idx] = B[idx*stride];
    }
}

int main() {

    int N = 1000;
    int stride = 8;

    double *d_A;
    cudaMalloc((void**) &d_A, N*sizeof(double)); // device array

    double *d_B;
    cudaMalloc((void**) &d_B, stride*N*sizeof(double)); // device array

    ...
    int threadsPerBlock = 256;
    int blocksPerGrid = N/threadsPerBlock + 1;

    nonCoalescentVectorAdd <<<blocksPerGrid, threadsPerBlock>>> (N, d_A, d_B);
    ...
}

```

Coalescent memory access to A array
Non-coalescent memory access to B array.

Coalescent array

Non-coalescent array

Multiple blocks parallel kernel

References

Wu, W. (2007). Computational River Dynamics. NetLibrary, Inc, CRC Press .

Fernández-Pato J., et al., 2025. Acceleration of pipeline analysis for irrigation networks through parallelisation in Graphic Processing Units. Biosystems Engineering 252, pp. 1-14.

An, S., and Seo, SC., 2020. Efficient Parallel Implementations of LWE-Based Post-Quantum Cryptosystems on Graphics Processing Units. Mathematics 8(10) pp. 1781.

NVIDIA developers, 2025. https://developer.nvidia.com/

CUDA Toolkit Documentation v12.8 Update 1, 2025. https://docs.nvidia.com/cuda/

Aragón Institute of Engineering Research I3A, 2025. https://i3a.unizar.es/es